

Date: 19/3/2026

Hamburg



Hochschule für Angewandte Wissenschaften Hamburg
Hamburg University of Applied Sciences

Design and Development of an Over-the-Air Programmable Standalone Dew Point Controller with a Web Interface

Main Developer: Trinh Minh Khoi Dinh

Supervisor:

- Dr. Veit Dominik Kunz
- Mr. Jan-Claas Böhmke

Acknowledgements

I would like to express my sincere gratitude to all those who supported me throughout the completion of this work.

First and foremost, I would like to thank my main supervisor: Dr. Kunz for his continuous guidance, valuable feedback, and encouragement during the development of this project. His insights and suggestions greatly contributed to improving both the technical quality and presentation of this work.

I would also like to thank Mr. Böhmke from Umwelttechnik und Elektronik laboratory for his helpful discussions and support. His contributions created a productive and motivating working environment.

Finally, I would like to acknowledge all the unanswering support from my family and my friends during the development of this project.

Trinh Minh Khoi Dinh
May 6, 2026

Contents

1	Introduction	3
2	Background	3
2.1	Product Requirements	5
3	Product	6
3.1	Hardware Configuration	6
3.2	Software Configuration	8
3.2.1	Dew Point Algorithm	8
3.2.2	Web Server Initialization	8
3.2.3	SD Card Logging	12
3.2.4	LittleFS, Flash Partition and Over-the-Air (OTA)	13
3.2.5	Realtime Operating System (RTOS) Implementation	14
3.3	Client-Side Web Interface Configuration	16
3.3.1	Main page (STH4x Dew Point Monitor)	16
3.3.2	Sub page (Over-the-Air (OTA) Update)	21
4	Result	22

List of Figures

1	Figure 1 Temperature, Relative Humidity and Dew Point [7]	4
2	Product Schematic	7
3	Flow chart of operation	11
4	LittleFS Illustration	13
5	Mutex Example [6]	15
6	Main Website Overview	16
7	OTA Website Overview	21
8	Display results during initialization.	22
9	Display result of the developed product and the final integrated prototype.	22
10	Website + Data.csv test	23

List of Tables

1	Overview of direct and indirect costs for different respiratory diseases [2]	3
2	Product tasks information	5
3	TTGO T-Display Datasheet	6
4	SHT4x Sensor Specifications	7
5	Wi-Fi Information	8
6	Flash Partition Layout	14
7	RTOS Tasks Distribution	14

Keyword: I²C , SPI, API, AC, DC, HTML, OTA, STA, AP, RTOS

1 Introduction

Maintaining a healthy indoor environment requires careful regulation of temperature and humidity, as excessive humidity can lead to mold growth, material degradation, and health issues. Conventional Heating, Ventilation, and Air Conditioning (HVAC) systems, while effective, are often costly, energy-intensive, and unsuitable for small-scale or low-budget applications. Recent advances in the Internet of Things (IoT) have enabled the development of low-cost, connected monitoring and control systems using microcontrollers and digital sensors. This project aims to design and implement an IoT-based Dew Point Controller system using the ESP32 microcontroller and SHT4x humidity/temperature sensors for indoor humidity regulation.

2 Background

Indoor humidity must be maintained within a balanced range to prevent both health risks and property damage. Prolonged high relative humidity increases the likelihood of condensation and mold growth, which is consistently associated with asthma and allergic rhinitis symptoms, bronchitis, and respiratory infections, especially for vulnerable populations [1]. A study by Urlaub and Grün et al. [2] in Europe further indicates that living in damp or moldy dwellings is associated with a 40% higher odds of asthma compared with non-damp homes, highlighting that moisture control is a practical preventive measure rather than only a comfort issue. Such mold/dampness-related respiratory outcomes can scale to a country-level burden through medical spending and lost productivity.

Table 1: Overview of direct and indirect costs for different respiratory diseases [2]

Illness	Direct costs [billions €]	Indirect costs [billions €]	Monetized value of DALYs lost [billions €]	Total costs [billions €]
COPD	23.3	25.1	93.0	141.4
Asthma	19.5	14.4	38.3	72.2
Pneumonia / Acute lower respiratory infections	2.5	NA	43.5	46.0

In addition to these health concerns, prolonged moisture exposure can damage property and increase renovation costs, which can become very expensive on a national scale. For example, a Finnish building-stock study estimated about € 7.7–8.0 billion in remediation costs, while the related health benefits were only about € 1.2 billion [3]. This issue becomes more critical during vacancy periods, when heating is reduced and uncontrolled ventilation can keep indoor relative humidity elevated (approximately 75–90%), creating persistently mold-favorable conditions. Hygrothermal simulation studies indicate that unacceptable mold risk can develop in as little as ~ 1.5 months depending on season, while window condensation may persist for months [4].

To address these issues, this work proposes an ESP32-based embedded system integrating dual SHT4x sensors to monitor both indoor and outdoor environmental parameters and maintain surrounding humidity within an optimal range. The system calculates dew point differences and leverages natural ventilation to control indoor humidity. The target operating condition is the “green zone,” corresponding to approximately 40–60% relative humidity (RH) at typical indoor temperatures of ~ 18 – 25 °C, which represents the most desirable range for comfort and reduced moisture risk (Figure below). Below this zone, the air becomes too dry, leading to irritation and discomfort; above this zone, the environment shifts toward condensation risk and mold-favorable conditions, particularly as RH approaches $\sim 70\%$.

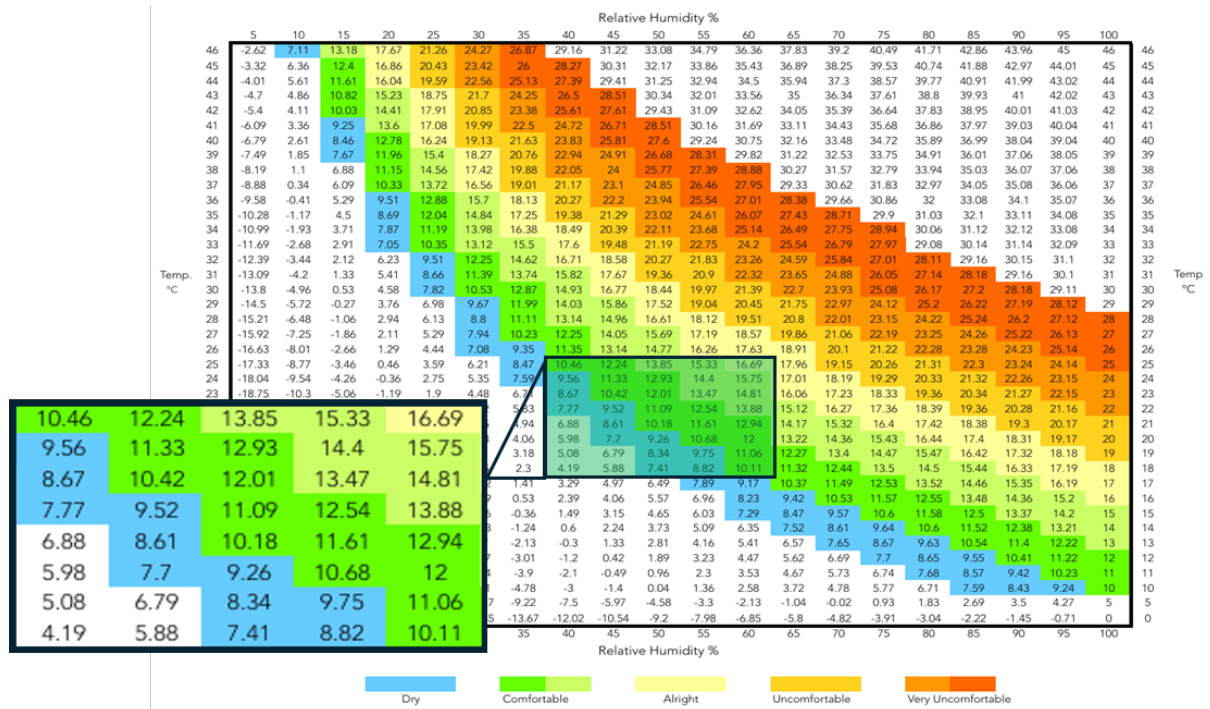


Figure 1: Figure 1 Temperature, Relative Humidity and Dew Point [7]

2.1 Product Requirements

The system combines sensing, data logging, web-based monitoring, and automatic control within a single embedded platform. It is designed as a compact and cost-effective solution for humidity regulation in small enclosed environments such as cottages or storage spaces. The following section outlines the specific requirements of the proposed system.

Table 2: Product tasks information

Requirements List	
Section	Tasks
Hardware	A custom EU Plug Housing Socket with Local Display via the TTGO ESP32 LCD
	Solid State Relay for Load control and SD Reader for History logging Installed
	Reading STH4x Sensors through 2 I2C ports + Display on ESP32 Screen
Client Side Software *	Main page Monitor with various Functions (remote reset, file CSV export ,...)
	Sub page for choosing either upload Hardware or Software update patch
ESP32 Core Software *	LittleFS Implementation + Flash Partition *
	Over the Air (OTA) for wireless updates possible
	Station (STA) and Access Point (AP) mode for Wi-Fi Connection *
	Realtime Operating System (RTOS) for task process *
	SD card logging operation following specific time interval *

Note: Extended descriptions for some tasks * above:

- The software of this product separate into 2 parts: software execute on client side and software executed on the ESP32 core
- Wi-Fi has flexible switching between STA and AP mode.
- LittleFS and Flash partition implementation for efficient file separation and organized memory usage.
- Real-time Operating System (RTOS) is already built into the Xtensa[®] dual-core processor of the TTGO T-Display (default core: 1).
- SD card reader saves all recorded data at 5-minute intervals per sample and automatically clears data after one year to free memory.

3 Product

3.1 Hardware Configuration

The system consists of:

- **The Crydom CX240D5R Solid State Relay (SSR)** is used to switch the 240 V AC fan load using a low-voltage DC control signal from the microcontroller. It accepts an input control voltage of 3–15 V DC, which makes it suitable for direct control from the ESP32-based TTGO T-Display. Pin 15 is used to control the load system through the SSR.
- **The SD card reader module** is used for local data logging and communicates with the ESP32 through the Serial Peripheral Interface (SPI).
- **The TTGO T-Display** is an ESP32-based development board that serves as the central controller of the monitoring and control system. It integrates a dual-core ESP32 microcontroller and a 1.14-inch TFT LCD display driven by the ST7789 controller, which can be programmed using graphic libraries such as `TFT_eSPI` and allows real-time visualization of system information. The board provides multiple communication interfaces, where I²C and SPI are used in this project. (For more information, see Table 3)

Table 3: TTGO T-Display Datasheet

Parameter	Specification
Main Processor	ESP32 dual-core LX6
CPU Frequency	Up to 240 MHz
Wireless Connectivity	Wi-Fi, Bluetooth
SRAM	520 KB
Flash Memory	4 MB
Operating Voltage	2.7-4.2V
GPIO Pins	Up to 34
Communication Interfaces	I ² C, SPI, UART, PWM, ADC, DAC

And finally, a 240VA/5V DC power supply to convert the input from ac socket as power source for TTGO T display

The STH4x sensor parts will be outside of the box so it can easily remove/attach for use, and it uses Inter-Integrated Circuit (I²C) interface to transmit the signal reading to ESP32. The general information on it is displayed below

Table 4: SHT4x Sensor Specifications

Parameter	Specification
Supply Voltage	2.4–5.5 V
Temperature Range	−40 to 125 °C
Temperature Accuracy	±0.2 °C
Humidity Range	0 to 100 %RH
Humidity Accuracy	±1.8 %RH

For overall view of wiring, the schematic below shows exactly how it looks inside the box

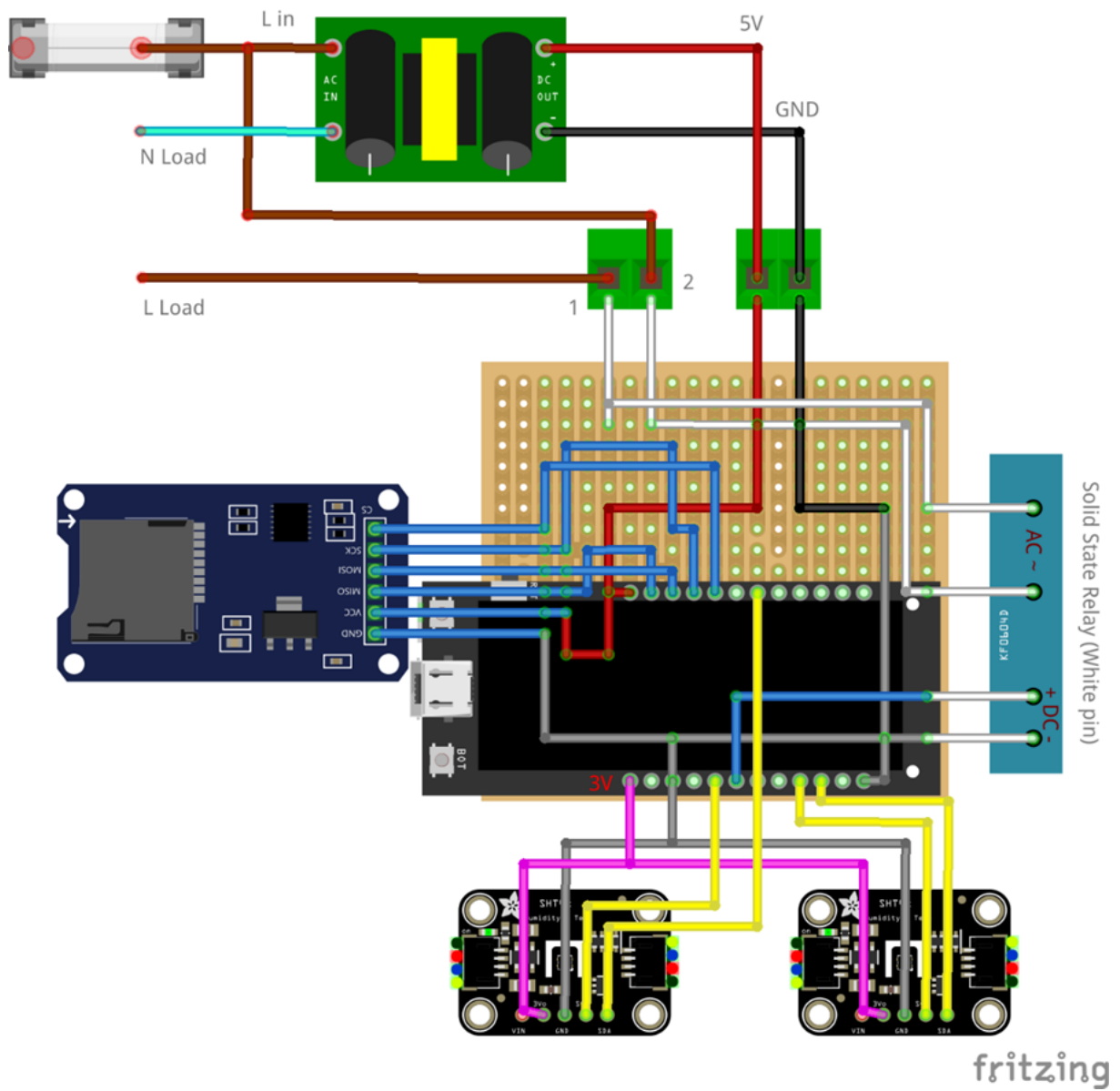


Figure 2: Product Schematic

3.2 Software Configuration

The software is developed in PlatformIO using the Arduino framework for the TTGO T-Display ESP32 board. The project uses LittleFS as the filesystem, a custom partition table (`my4MB_OTA.csv`), and several external libraries, including `TFT_eSPI`, `Adafruit_SHT4x`, `AsyncTCP`, and `ESPAsyncWebServer`. (`platformio.ini` file)

3.2.1 Dew Point Algorithm

The ESP32 will display these following parameters on the screen:

- Temperature and Humidity of both sensors
- Dew point calculation following the formula based on the Magnus-Tetens approximation [5]:

$$\gamma(T, RH) = \frac{aT}{b + T} + \ln \left(\frac{RH}{100} \right) \quad (1)$$

$$T_d = \frac{b \cdot \gamma}{a - \gamma} \quad (2)$$

where RH is the relative humidity (in %), T is the air temperature ($^{\circ}\text{C}$), and a and b are empirical constants (commonly, $a = 17.27$ and $b = 237.7^{\circ}\text{C}$, respectively).

During the automated measurement process, which is performed at 5-minute intervals, the fan turns on when ($\gamma > 5$) and turns off when ($\gamma \leq 2$).

3.2.2 Web Server Initialization

The web server operates asynchronously with either of 2 statuses: Station (STA) and Access Point (AP) during the ESP32 initialize and contains following various functions to interact:

- _ Select day and download CSV as ZIP or single download
- _ Manual disable fan during running and clear chart
- _ Reboot button (Red) for remote reset device
- _ Chart display on both sensors and other parameters

Note: The ESP32 SD logging and time record only operates on STA mode (credentials below), otherwise it will run as AP mode with the same info as below. Domain in STA mode can be variable on device. Moreover, with the support of dynamic switching run in background, the device can automatically connect whenever network is available without needing restart.

Table 5: Wi-Fi Information

ID (Wi-Fi Name)	STH4x_Monitor
Password	12345678
Domain (AP mode)	192.168.4.1
Domain (STA mode)	10.25.218.241

To ensure controlled SD data logging and browser-based access, the ESP32 web server organizes its communication into several HTTP routes instead of handling all functions in a single endpoint. This separation allows time initialization, real-time monitoring, and SD card file access to operate independently without interfering with one another.

In this system, three main route groups are used:

- **Start Logging Route** (`/startLogging`): receives the current Unix epoch time from the browser when the monitoring page is opened. The ESP32 sets the internal system clock and calls and creates or opens the daily folder in the format `/DD-MM-YYYY/` and prepares the file `Data.csv`. After successful initialization, the logging flag is enabled so that the RTOS-based SD task can begin storing one sample every 5 minutes.
- **JSON Data Route** (`/data`): provides the latest measurement values and device status to the browser in JavaScript Object Notation (JSON) format. This route returns temperature, relative humidity, and dew point values from both SHT4x sensors, together with the fan state and manual disable condition. By repeatedly requesting this route, the web interface can refresh charts and current monitoring values in real time.
- **SD Card Routes** (`/sd/days`, `/sd/files`, `/sd/raw`, `/sd/status`): allow the browser to access stored measurement data on the microSD card.
 1. `/sd/days` lists available date folders,
 2. `/sd/files` shows the file names inside a selected folder,
 3. `/sd/raw` streams the selected CSV file for download,
 4. `/sd/status` reports the remaining time before the automatic yearly purge together with the warning status.

However, there is a small limitation happening during the SD logging operation which is:

- **Time Synchronization for Correct SD Logging**

The ESP32 does not maintain real-time clock information after rebooting. As a result, the internal time may fall back to an invalid epoch, which can cause incorrect folder names and timestamps during SD card logging. To avoid this problem, the proposed system applies a hybrid time-synchronization strategy.

When the device successfully connects in STA mode, it first attempts to synchronize its clock from the German NTP server `ptbtime1.ptb.de` which is more reliable reference than local clock from browser. (more info go to <https://www.ntppool.org/de/zone/de>)

```
// tryNtpSync: try to sync time via NTP, returns true if successful within timeout  
const char* NTP_SERVER = "ptbtime1.ptb.de";  
static void tryNtpSync(){
```

If NTP synchronization is not available, such as in AP mode or when internet access is unavailable, the system falls back to browser-assisted time initialization through the /startLogging route.

```
time_t epoch = (time_t) strtoll(req->getParam("epoch", true)->value().c_str(),  
    nullptr, 10);  
if (epoch < 1700000000) {  
    req->send(400, "application/json", "{\"ok\":0,\"err\":\"bad epoch\"}");  
    return;  
}  
setTimeFromEpoch(epoch);
```

The threshold of 1700000000 acts as a basic sanity check to reject clearly invalid timestamps. Only when the received value is considered valid does the ESP32 update its internal clock using setTimeFromEpoch(epoch).

• Prevention of Timer Overflow in Long-Term Operation

The ESP32 provides the millis() function as a 32-bit unsigned counter, which overflows after approximately 49.7 days. If time comparisons are implemented incorrectly, this overflow may lead to incorrect sampling intervals or unstable program behavior.

In the present system, the measurement period is defined explicitly as:

```
static const uint32_t MEAS_PERIOD_MS = 5UL * 60UL * 1000UL;
```

Which can be explained as:

$$\text{MEAS_PERIOD_MS} = 5 \text{ minutes} \times 60 \text{ seconds} \times 1000 \text{ milliseconds} = 300000 \text{ ms}$$

The flow chart of operations on next page will summary how the product fully functions all this section process.

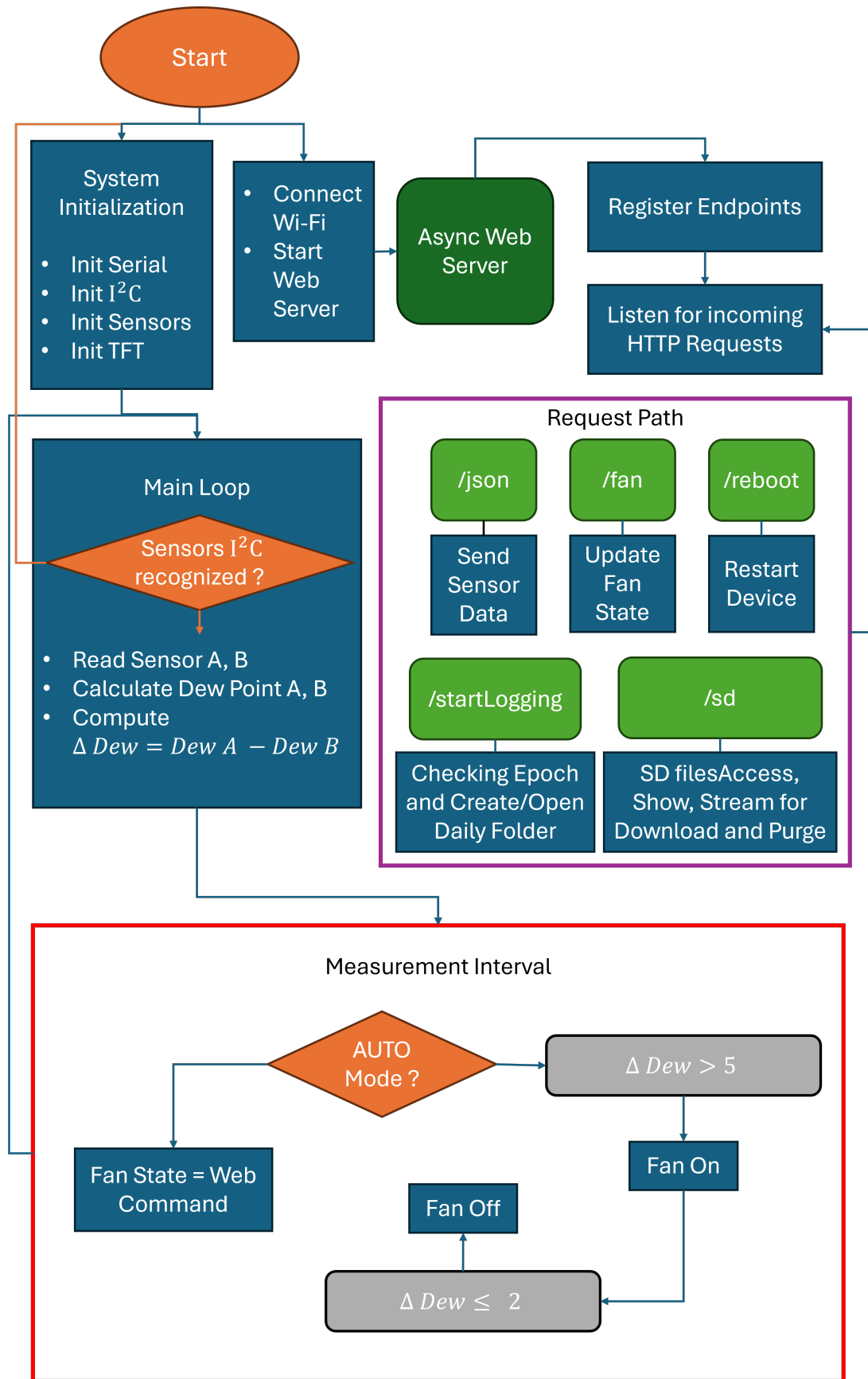


Figure 3: Flow chart of operation

3.2.3 SD Card Logging

The microSD logging module is designed as a self-contained component responsible for storing measured environmental data in CSV format. It operates independently from the LittleFS web-storage system (details below) and uses the external SD card over Serial Peripheral Interface (SPI) communication.

The steps of operation on self-contained SD logging Application Programming Interface (APIs) are as follows:

(Note: can be found in `sd_logger.h` and `sd_logger.cpp`)

- **SD card initialization.**

The logging process starts with `sdLoggerBegin()`, which initializes SPI communication and mounts the SD card. If this step succeeds, logging can continue normally. If the SD card is missing or initialization fails, all later logging operations are skipped safely.

- **Time validation before logging.**

The logger requires a valid system time before creating any file, since both the folder name and each CSV timestamp depend on the local time. Therefore, logging only starts after the ESP32 receives the correct epoch time from the web interface. Once time is valid, `sdLoggerStartForToday()` opens the logging file for the current date.

- **Daily folder and file creation.**

The module stores data in a date-based folder using the format `/DD-MM-YYYY/`, with the file name fixed as `Data.csv`. If the folder or file does not exist, it is created automatically, and the CSV header is written first. This keeps the logged data organized by day.

- **Writing one data row.**

Whenever a new measurement sample arrives, `sdLoggerWriteRow()` converts it into one semicolon-separated CSV line and appends it to the current file. Each row includes the local timestamp, sensor A and B values, dew-point difference, fan state, and disable state. The file is then flushed so the data is saved immediately.

- **Automatic day rotation.**

The module checks whether the date has changed during operation. If a new day is detected, the current file is closed and a new folder and `Data.csv` file are opened automatically. This ensures that each file contains data from only one day.

- **Yearly purge logic.**

The logger `sdLoggerGetPurgeInfo()` calculates the next January 1st at 00:00 local time as the purge point. When that time is reached, all SD card contents are deleted recursively through `sdLoggerPurgeIfDue()`. It can also report the remaining days before purge and raise a warning within the last three days so the browser can notify the user.

The data line, using semicolon as the delimiter in `Data.csv`, appears as follows:

iso_time	tempA	rhA	dewA	tempB	rhB	dewB	deltaDew	fan	disabled
----------	-------	-----	------	-------	-----	------	----------	-----	----------

Note: A critical design assumption is that the system time must already be valid before starting logging and writing, so the ESP32 must first connect through the web server in STA mode. Also, an example timeline of SD card operation is shown below:

```
00:00 -> Create daily.csv
00:05 -> Sample 1
00:10 -> Sample 2
...
23:55 -> Sample 288
00:00 -> New day -> New folder + daily.csv
```

3.2.4 LittleFS, Flash Partition and Over-the-Air (OTA)

For maintenance and scalability, LittleFS is implemented to separate web resources from the main firmware. This allows web interface components such as JavaScript and HTML files to be updated independently without modifying the core firmware logic. As a result, the dashboard structure and layout are easier to manage and maintain.

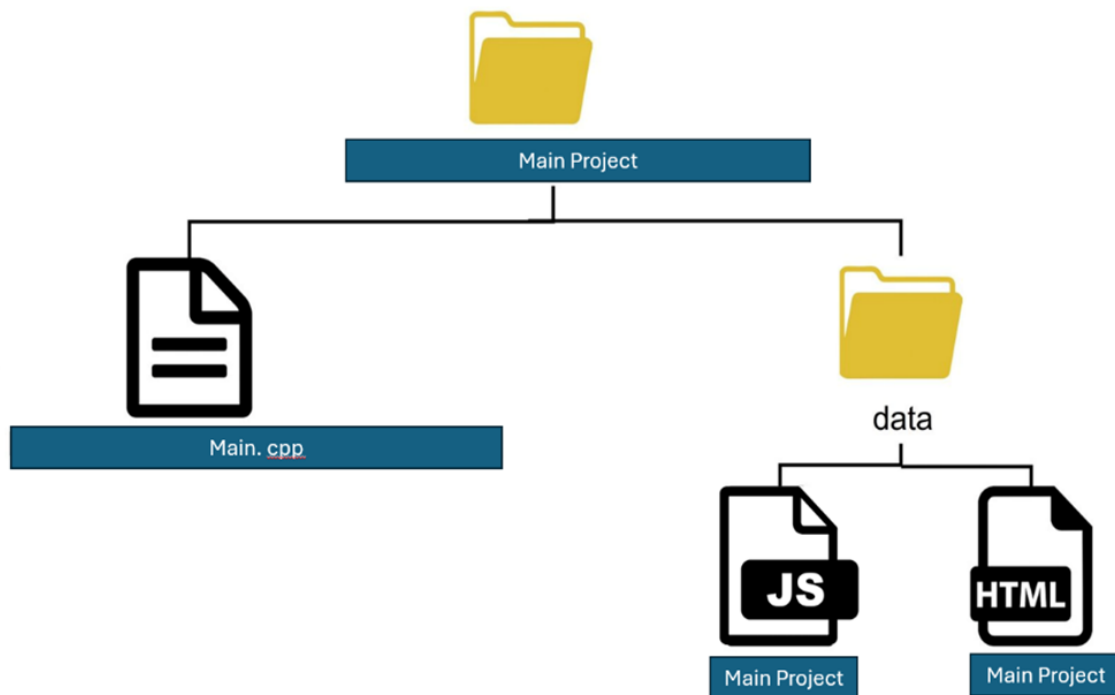


Figure 4: LittleFS Illustration

(Note: LittleFS initialization can be found in `main.cpp` using `LittleFS.begin()`.)

Moreover, using the Over-the-Air (OTA) method, system updates can be performed wirelessly through the web interface. The system supports uploading two types of binary files: `firmware.bin` for updating the main application and `littlefs.bin` for updating web files stored in LittleFS. To distinguish between these update types, the system uses separate handlers for firmware and filesystem updates.

To support both firmware and filesystem updates, the FLASH memory must be partitioned appropriately to optimized 4MB memory usage of ESP32 core instead of using the default configuration. The partition layout used in this project is shown in Table 6.

Table 6: Flash Partition Layout

Partition Name	Type	SubType	Offset	Size (Hex)	Readable Size	Purpose
nvs	data	nvs	0x9000	0x5000	20 KB	Stores Wi-Fi credentials, calibration values, and system settings
phy_init	data	phy	0xF000	0x1000	4 KB	RF calibration data for Wi-Fi and Bluetooth
app0 (ota_0)	app	ota_0	0x10000	0x1E0000	~1.875 MB	Main firmware slot (<code>firmware.bin</code>)
spiffs (LittleFS)	data	spiffs	0x1F0000	0x200000	2.0 MB	Filesystem for web UI (<code>index.html</code> , <code>app.js</code> , OTA files)
unused	–	–	0x3F0000	0x10000	64 KB	Remaining unused flash space

(Note: OTA implementation can be found in `ota_update.h` and `ota_update.cpp`.)

3.2.5 Realtime Operating System (RTOS) Implementation

The TTGO T-Display features two independent CPU cores (0 and 1). By default, in the ESP32 system, the main application (i.e., `setup()` and `loop()`) runs on core 1. In this project, the program is divided into multiple FreeRTOS tasks and distributed across both cores, allowing sensing, display updates, and SD card logging to execute concurrently without blocking.

In this system, three main tasks are created:

- **TaskSensor:** Reads SHT4x sensors, computes dew point and dew-point difference, controls the fan logic, and generates one data sample every 5 minutes.
- **TaskSD:** Receives data from a queue and writes it to the SD card in CSV format.
- **TaskDisplay:** Updates the TFT display periodically 5 minutes with the latest sensor values and system status.

Table 7: RTOS Tasks Distribution

Task	Core	Priority
TaskSensor	1	2
TaskSD	1	1
TaskDisplay	0	1

A shared state structure protected by a mutex is used to safely exchange data between tasks. This ensures that only one task can access shared variables at a time, preventing data corruption.

```
// Shared reading state
struct SharedState {
    ... };
static SharedState g_state;
static SemaphoreHandle_t g_stateMutex = nullptr;
```

To protect access (which does not make shared value between these tasks being overwritten in some case loop action happen), it uses a mutex (mutual exclusion) which operate like a lock that allows only one task at a time to access shared data (`g_stateMutex`)

- If a task holds the lock, other tasks must wait.
- When the task finishes, it releases the lock.
- Another task can then access the shared resource.

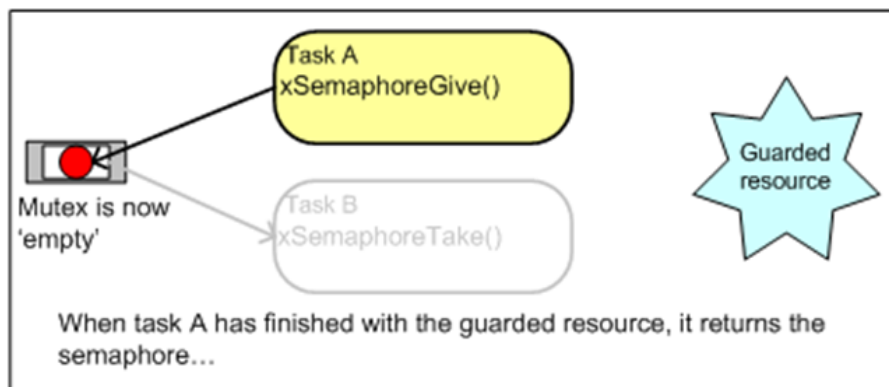
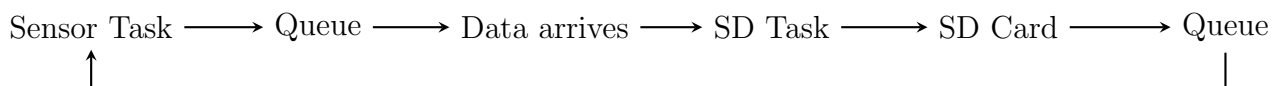


Figure 5: Mutex Example [6]

To decouple sensing from SD writing, a FreeRTOS queue is used between TaskSensor and TaskSD. The sensor task pushes data into the queue, and the SD task consumes it for storage. This avoids delays caused by slow SD card operations.

Task execution flow:



```
static QueueHandle_t g_sdQueue = nullptr;

g_sdQueue = xQueueCreate(4, sizeof(SDLogSample));

// TaskSensor produces data
xQueueSend(g_sdQueue, &s, 0);

// TaskSD consumes data
xQueueReceive(g_sdQueue, &s, portMAX_DELAY);
```

The measurement interval is controlled in the sensor task using:

```
vTaskDelay(pdMS_TO_TICKS(MEAS_PERIOD_MS));
```

3.3 Client-Side Web Interface Configuration

3.3.1 Main page (STH4x Dew Point Monitor)

On the client side, the monitoring interface is implemented as a browser-based web application served by the ESP32. In this system, the ESP32 acts as a web server with layout as HyperText Markup Language (HTML), while the browser acts as a client that receives, executes, and renders the interface in JavaScript (JS). The client-side implementation consists of two main components: the monitoring dashboard (`index.html` + `app.js`) and the OTA update page (`ota.html`) and those connected to `main.cpp` through routes (see Web Server Initialization).

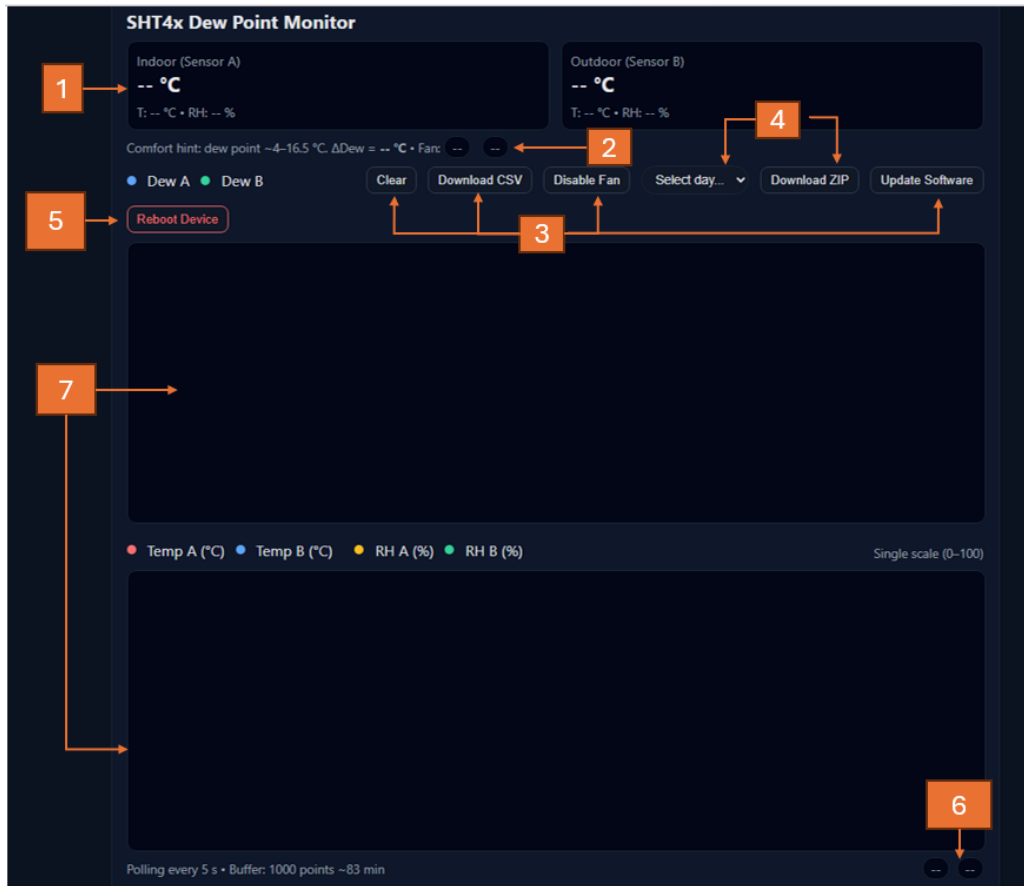


Figure 6: Main Website Overview

Main container (appRoot): This is the root block of the dashboard. It contains all main interface parts, including the sensor panels, control buttons, and charts. JavaScript can access this container through `document.getElementById('appRoot')` and hide the whole interface when needed, for example during reboot.

JS

```
const appRoot =  
document.getElementById('appRoot');  
appRoot.style.display = 'none';
```

HTML

```
<div class="card"  
id="appRoot">
```

The functionality of each numbered component is explained as follows:

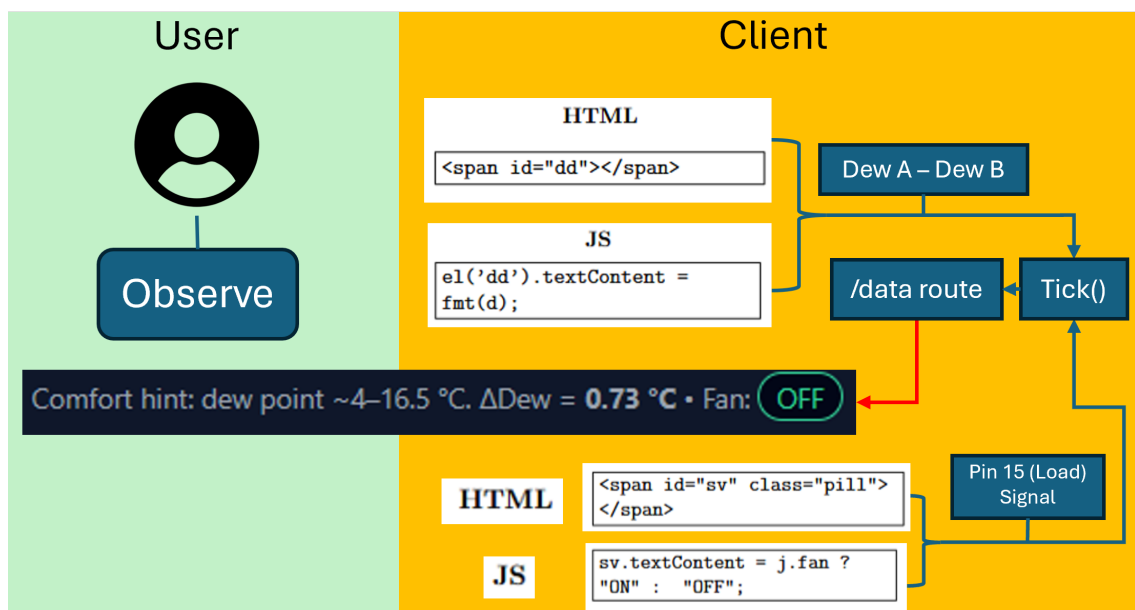
1. **Sensor data Readings:** These zone display the real-time monitoring values for Indoor (Sensor A) and Outdoor (Sensor B). Each zone presents the dew point as the main value, followed by temperature and relative humidity. The function `tick()` is used to send request `/data` route which provided by and update UI. (This function will repeatedly use for the rest which need UI update).



2. **Status summary line:** This line displays the dew-point difference (ΔDew) and the fan state.

The ESP32 provides sensor values through the `/data` route, including `dewA`, `dewB`, and `fan`.

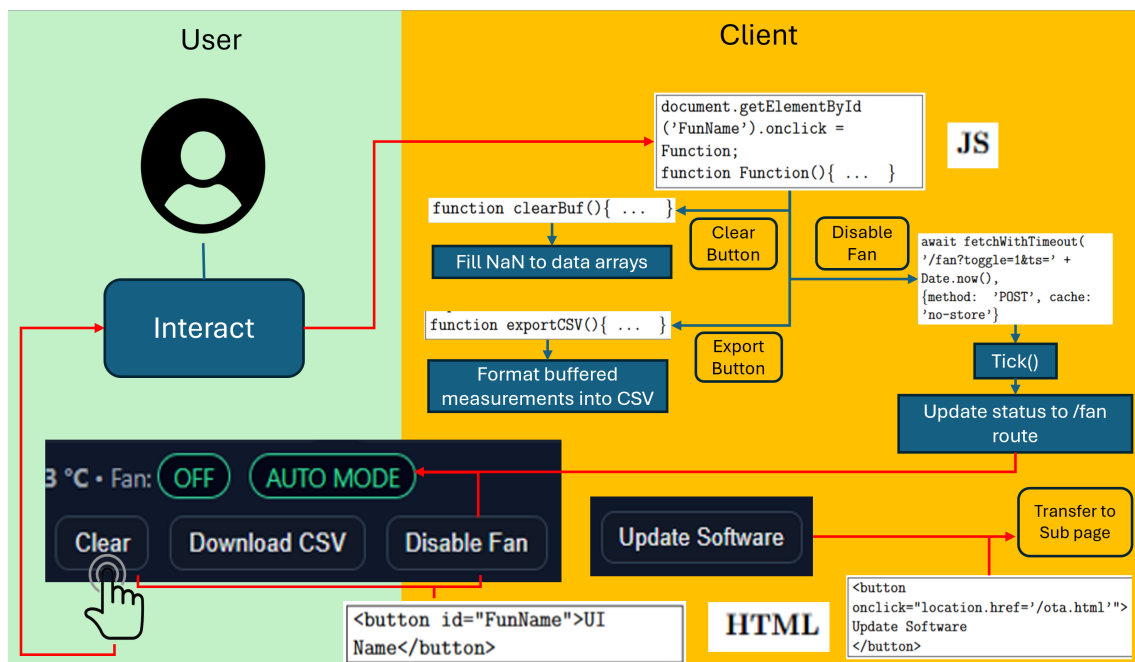
On the client side, ΔDew is computed in JavaScript as the difference between `dewA` and `dewB`, and then updated in the HTML using `textContent`. The fan state is determined on the ESP32 based on the control logic and sent to the client, where it is displayed as an ON/OFF indicator.



3. Various Interaction Buttons:

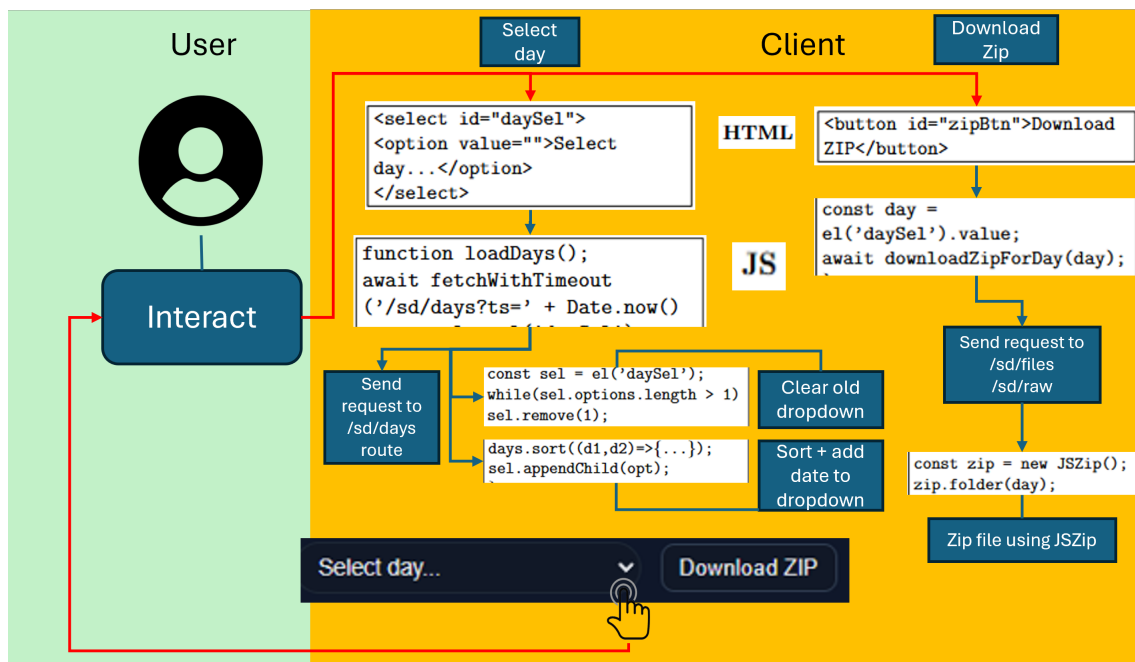
Each button is defined in HTML using `<button>` elements. In most cases, the click behavior is attached in JavaScript through a unique `id` and execute function by using `document.getElementById(...)`, while the Update Software button uses an inline `onclick` handler with `location.href` to redirect the browser to another HTML page. When pressed, the button either performs a local browser action or sends a request to an ESP32 route (Fan Disable).

- **Clear button:** The click event is handled in JavaScript to process `clearBuf()` which fills all buffer elements with `NaN` to resets all locally stored plotting buffers and redraws the charts.
- **Download CSV button:** The click event is handled in JavaScript to execute function `exportCSV()` where all the browser-side buffered data converts into a CSV file and triggers the download.
- **Disable Fan button and Auto Mode pill:** When pressed, JavaScript sends a request to the ESP32 using `fetchWithTimeout('/fan?toggle=1', {method: 'POST'})`, and then calls `tick()` to retrieve the updated system state. The function `setFanUI()` subsequently updates the mode pill to display either AUTO MODE or MANUAL OFF, depending on the value of `fanDisabled`.
- **Update Software button:** When pressed, no request to the ESP32 is sent immediately. Instead, the browser performs a local page redirection to `/ota.html`, which opens the OTA update page stored in LittleFS. From there, the user can upload either a firmware binary or a LittleFS image (Explain later in second section)

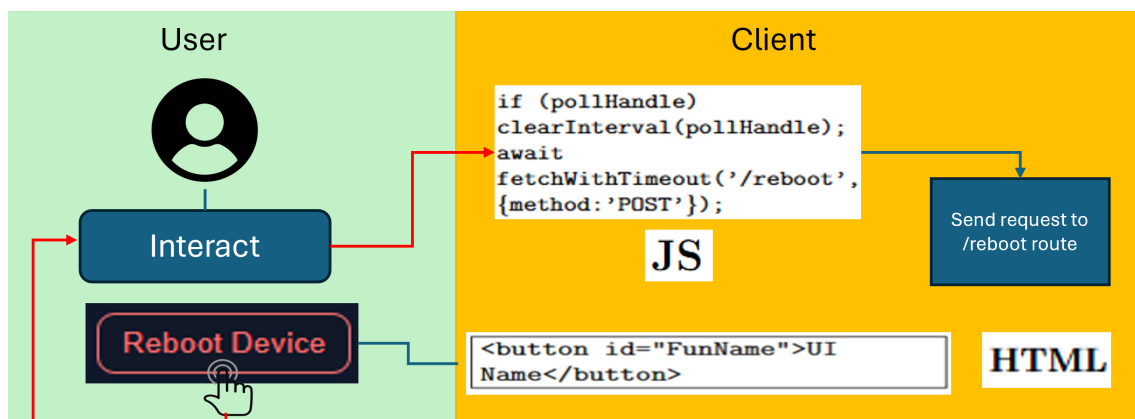


4. Day selection dropdown and Download Zip:

- **Day selection dropdown:** The dropdown is filled dynamically using the available folder names returned by the `/sd/days` route. It allows the user to select a specific day of stored SD card data. The function `loadDays()` requests the available dates, clears old entries, and appends new `<option>` elements into the selector.
- **Download ZIP button:** This button downloads all files from the selected day. It retrieves file names and contents from the ESP32 using the `/sd/files` and `/sd/raw` routes, and then packages them into a ZIP file in the browser using the JSZip library. The click event triggers `downloadZipForDay(day)`, which fetches the file list, reads each file, and adds them into a ZIP archive.

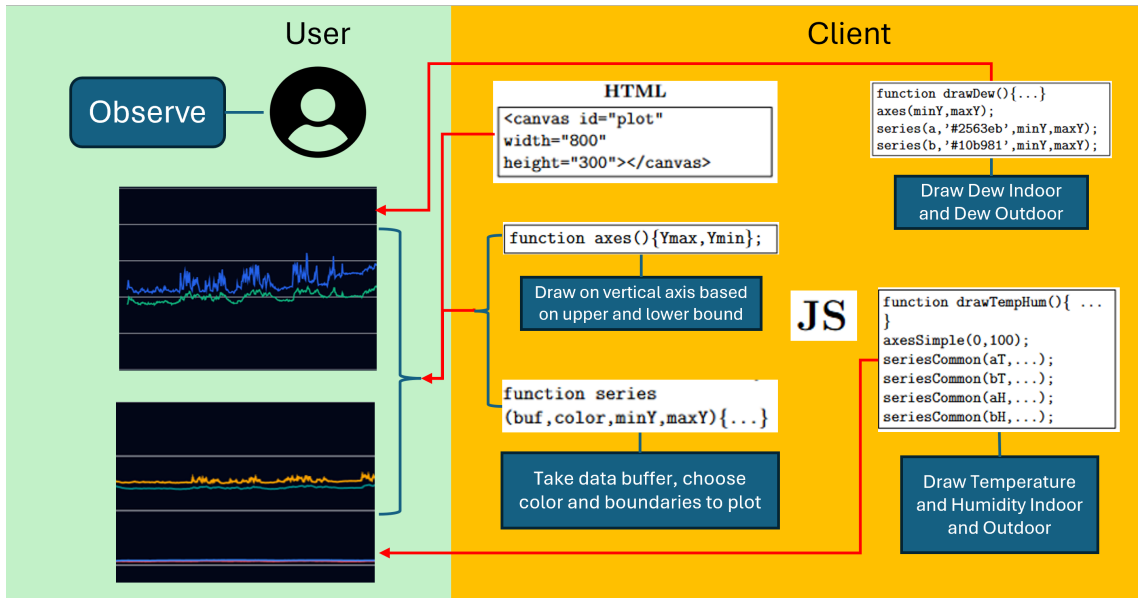


5. **Reboot Device button:** This button follows the same mechanism as the previous buttons (Previous Page) and it triggers a system reboot through the `/reboot` route. When pressed, it first displays a reboot message overlay, stops the periodic polling process, and then sends a POST request to the ESP32. After the request is issued, the browser clears the interface while the ESP32 reboots.

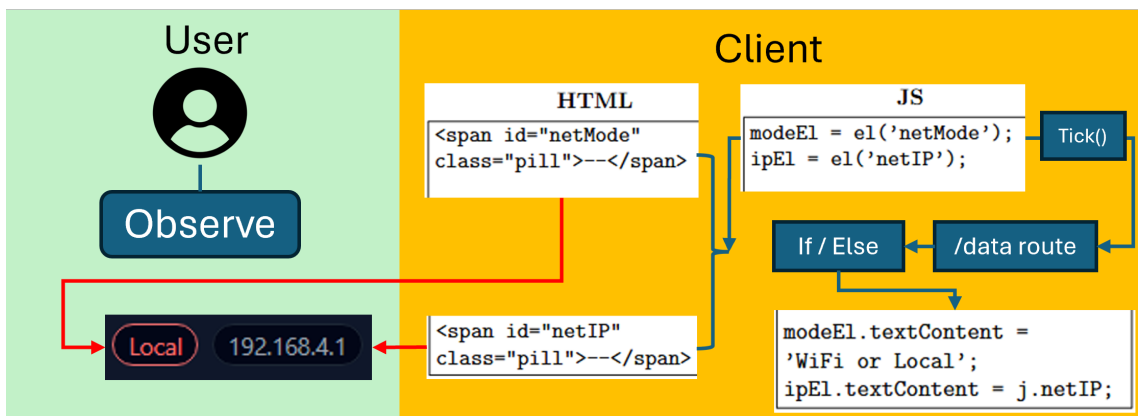


6. **Chart display components:** The chart area of the web interface is implemented using HTML `<canvas>` elements, which provide a drawable region for browser-side visualization. In JavaScript, each canvas is first accessed through its element id and associated with a 2D drawing context, such as `const ctx = cvs.getContext('2d');`. The plotting buffers are then updated periodically using local data already got form `/data` route and redrawn dynamically without needed `tick()`.

- **Dew-point chart:** This chart is displayed in the canvas element `plot` and visualizes the historical dew-point values of Sensor A and Sensor B. In JavaScript, the function `drawDew()` is responsible for redrawing the chart, while `axes()` renders the vertical scale and `series()` plots the two dew-point curves.
- **Temperature and humidity chart:** This chart is displayed in the canvas element `thPlot` and visualizes the temperature and relative humidity values of both sensors. In JavaScript, the function `drawTempHum()` redraws the chart using a shared vertical axis from 0 to 100, while `axesSimple()` and `seriesCommon()` are used to render the axis and the four data series.



7. **Network status indicators:** These indicators show the current network mode and IP address of the ESP32. The values are obtained from the `/data` route and updated periodically in JavaScript, where the elements `netMode` and `netIP` are modified according to whether the system is operating in STA or Local AP mode.



3.3.2 Sub page (Over-the-Air (OTA) Update)

In addition to the **Update Software** button on main dashboard, the system provides a separate OTA update interface as shown in Figure 7.

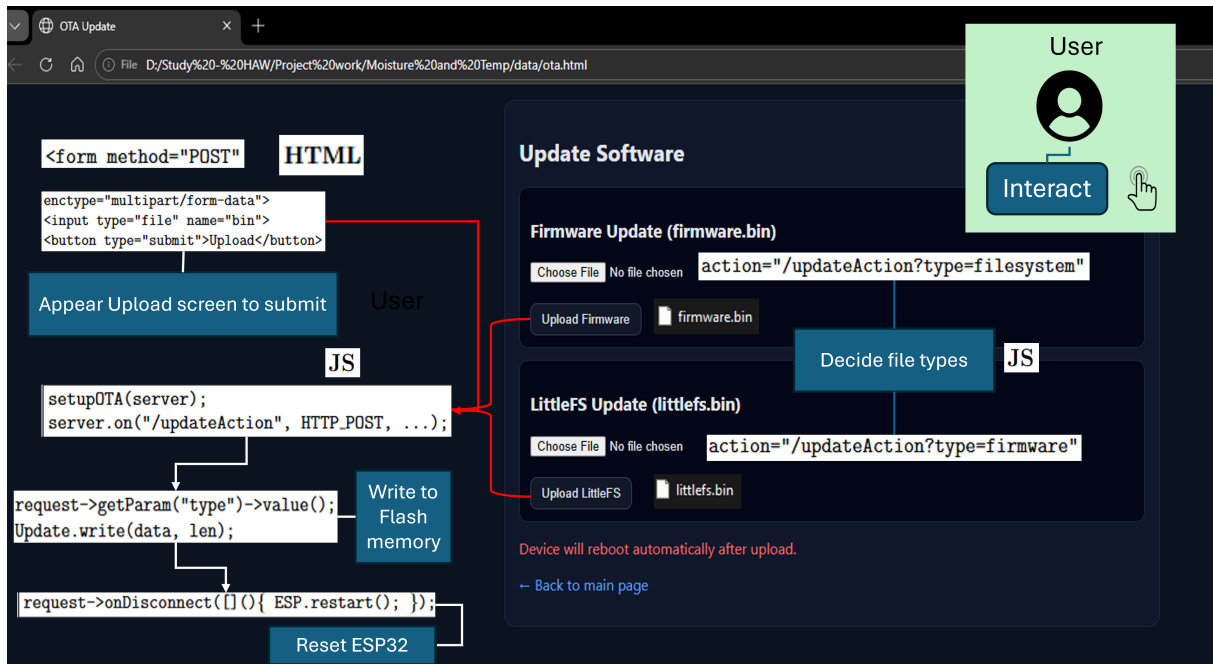


Figure 7: OTA Website Overview

The OTA page allows users to upload firmware and filesystem images through HTTP POST forms. When the user selects a binary file and presses the upload button, the browser sends the file to the `/updateAction` route with a corresponding type parameter (`firmware` or `filesystem`).

On the ESP32 side, the function `setupOTA(server)` defines this route and handles the incoming upload. The parameter `type` is first extracted using `request->getParam("type")` to determine the target partition. The uploaded file is received in chunks and written to flash memory using `Update.write(data, len)`.

After the upload is completed successfully, the ESP32 sends a response to the browser and then restarts automatically using `ESP.restart()` in `ota_update.cpp`, applying the updated firmware or filesystem.

4 Result

During the initialization, the product will display if it can read the sensor or not. Then, after 5 seconds, the Wi-Fi establish whenever Station (STA) or Access Point (AP) mode with their own unique ID for entering on the web domain.

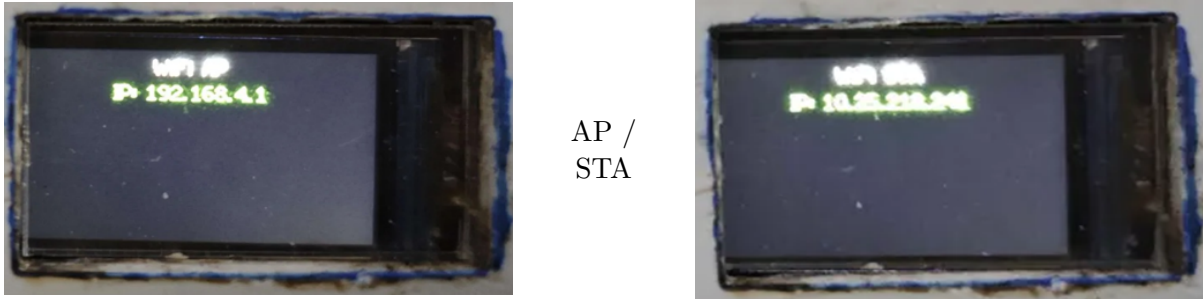


Figure 8: Display results during initialization.

After initialization, the full readings from both sensors are shown on the display using different colors for easier identification. Under normal dew-point conditions, the bottom status line appears in black or green. When the dew-point difference exceeds the defined threshold, the fan is activated and the status line changes to red. The fan remains in this state until the next measurement interval confirms that the environment has returned to a safe humidity condition.

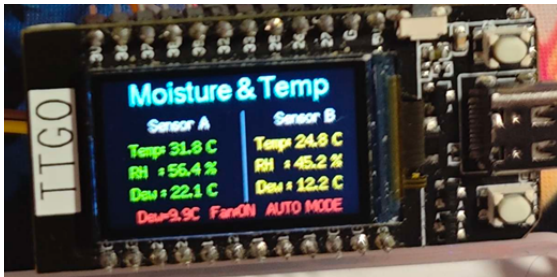
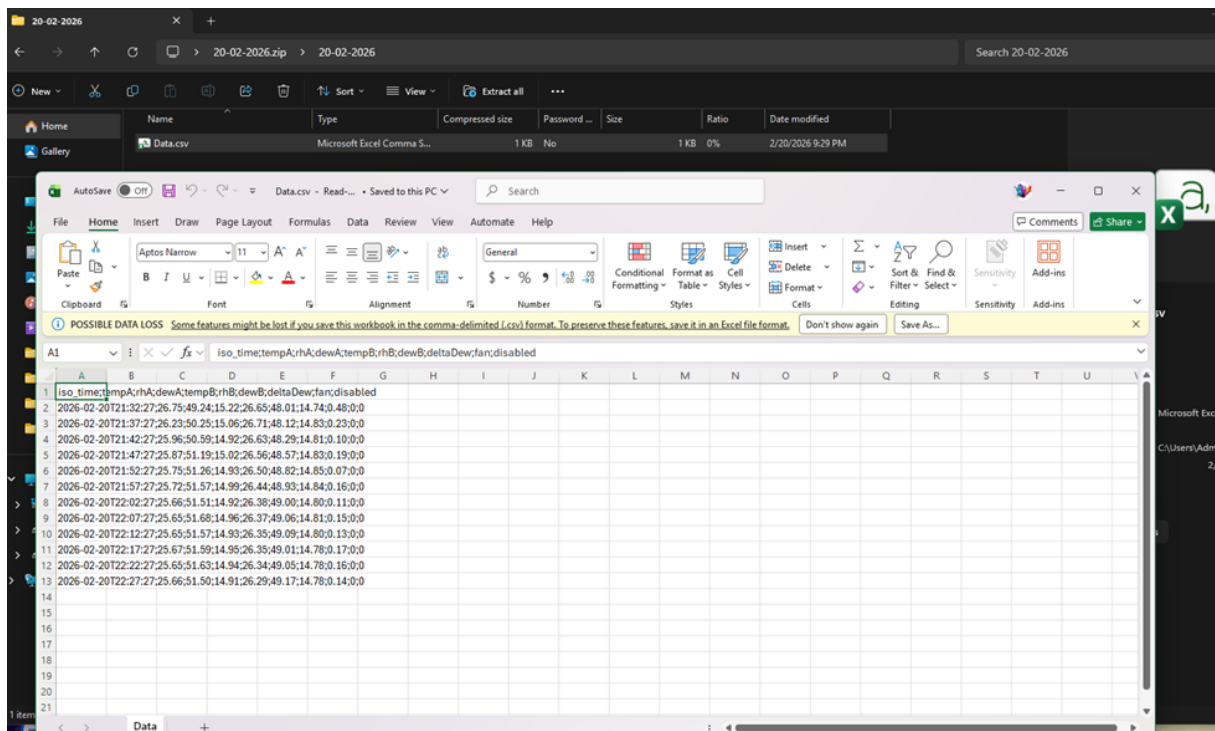


Figure 9: Display result of the developed product and the final integrated prototype.

On the web server, the signal transmitted from the ESP32 appears with a delay of less than one second and matches the physical display. The “Disable Fan” button works as expected, updating the website state and controlling the fan simultaneously. When the CSV data file is downloaded to the computer, the recorded parameter values are consistent with those shown in the website charts. In addition, all other functions operate correctly and reliably.



Figure 10: Website + Data.csv test



References

- [1] J. Hurraß et al., “Indoor Mold—Important Considerations for Medical Advice to Patients,” *Deutsches Ärzteblatt International*, 2024
- [2] S. Urlaub and G. Grün, “Mould and dampness in European homes and their impact on health,” *Fraunhofer Institute for Building Physics (IBP)*, IBP-Report EER-058/2016/950, 2016.
- [3] Hörhammer et al., “Remediating moisture- and mould-damaged residential buildings in Finland – Costs and benefits for health and climate,” *Indoor Environments*, vol. 2, no. 2, 100094, 2025.
- [4] *Journal of Building Engineering*, 2024
- [5] M. G. Lawrence, “The relationship between relative humidity and the dewpoint temperature in moist air: A simple conversion and applications,” *Bulletin of the American Meteorological Society*, vol. 86, no. 2, pp. 225–234, Feb. 2005. doi: 10.1175/BAMS-86-2-225.
- [6] FreeRTOS, “Mutexes,” *FreeRTOS Documentation*. [Online]. Available: <https://www.freertos.org/Documentation/02-Kernel/02-Kernel-features/02-Queues-mutexes-and-semaphores/04-Mutexes>. [Accessed: Mar. 19, 2026].
- [7] Mr Fixit Bali, “Temperature, humidity and dew point,” *MrFixitBali*. [Online]. Available: <https://www.mrfixitbali.com/air-conditioning/temperature-humidity-dew-point-251.html>. [Accessed: Mar. 19, 2026].